

ULTRA DEEP-DIVE: Phishing Detection Project

Exhaustive Interviewer Q&A — "E2E ML Apps — Phishing Detection" Resume Bullet

Goal: no question should be able to surprise you on this project.

HOW THIS IS ORGANIZED

1. Business/Problem Framing Questions
 2. Data & MongoDB Questions
 3. Data Validation & Drift Questions (KS-test)
 4. Data Transformation Questions (KNN Imputer, label remapping)
 5. Model-by-Model Deep Dive (Random Forest, Decision Tree, Gradient Boosting, Logistic Regression, AdaBoost — why/where each is used)
 6. **The $R^2 \rightarrow F1$ Bug — Full Story, Resolved** (this is now one of your strongest talking points)
 7. MLflow / Dagshub Experiment Tracking Questions
 8. FastAPI / Deployment / CI-CD Questions
 9. Adversarial / Stress-Test Questions
 10. Numbers You Must Have Cold
-

1. BUSINESS / PROBLEM FRAMING QUESTIONS

Q1. "Why does phishing detection matter as a business problem — what's the cost of getting it wrong?" A. Two failure directions, each costly differently: a **false negative** (missing a real phishing URL) exposes users/organizations to credential theft, fraud, or malware; a **false positive** (flagging a legitimate site as phishing) breaks user trust and blocks legitimate traffic. In most security contexts, false negatives are considered more costly — which is exactly why metric choice (F1/recall over raw accuracy) matters, a theme that runs through this entire project.

Q2. "Who would use this system in practice — an end user, a browser, a security team?" A. Realistically, this kind of model sits behind a service (browser extension, email gateway, or corporate proxy) that checks a URL/network-feature signature in real time before a user reaches a page. My implementation exposes it as a FastAPI `/predict` endpoint, which is the natural integration point for that kind of downstream consumer.

Q3. "What's the actual input to the model — the URL string itself, or something else?" A. Be precise here based on your dataset's actual columns: this dataset uses **network/URL-derived features** (e.g., things like URL length, presence of `@` symbol, use of HTTPS,

domain age, redirect count — check your actual `schema.yaml` for the exact feature list) rather than raw text/NLP on the URL string itself. This is a classic structured/tabular classification problem, not an NLP problem — important distinction if asked "did you use NLP techniques" — the honest answer is no, this is feature-engineered tabular data.

► **Will dig in:** if you can't name 3-4 actual feature columns from your dataset, that's a gap — go check `schema.yaml` before your interview.

2. DATA & MONGODB QUESTIONS

Q4. "Why MongoDB instead of a SQL database or flat CSVs?" A. The dataset is tabular/flat by nature, so SQL would work equally well here — I chose MongoDB primarily to get hands-on NoSQL/cloud-database exposure, and because phishing feature sets tend to evolve over time (new URL/network indicators get added), so a schema-flexible document store avoids migration overhead as features change.

► **Will dig in:** "Doesn't that undercut your Data Validation schema check?" → No — it's the opposite: the schema check *replaces* the structural guarantee a SQL table would give you for free, since Mongo doesn't enforce one.

Q5. "Walk me through the ETL — how does data go from CSV to MongoDB to your pipeline?" A. Extract: raw phishing CSV → local file. Transform: `push_data.py` converts each row into a JSON document (`cv_to_json_converter()`). Load: `insert_data_mongodb()` inserts those JSON records into a MongoDB collection. Later, `export_collection_as_dataframe()` in `data_ingestion.py` pulls that data back out as a Pandas DataFrame, drops the auto-generated `_id` column, and converts NA markers to proper NaN values before splitting into train/test.

Q6. "Why store the Mongo URI in `.env` instead of hardcoding it?" A. Security best practice — credentials should never be committed to version control. `.env` is git-ignored, and the URI is loaded at runtime via environment variables, so the actual password/connection string never appears in the GitHub repo history.

3. DATA VALIDATION & DRIFT QUESTIONS

Q7. "What is data drift, and how did you detect it?" A. Drift = the statistical distribution of new/incoming data differs meaningfully from the data the model trained on, meaning the model's assumptions may no longer hold. I used the **Kolmogorov-Smirnov (KS) test**, comparing each column's distribution in the training set vs. test set — a low p-value indicates the two samples likely come from different distributions (drift detected).

Q8. "What's the null hypothesis of the KS-test, and what threshold did you use?" A. Null hypothesis: the two samples come from the same underlying distribution. A p-value below your threshold (conventionally 0.05) rejects that null → drift flagged. **Check your actual**

code/constants file for your exact threshold before the interview — don't guess a number you haven't verified.

Q9. "Does KS-test work on categorical features?" A. No — KS-test is designed for continuous numerical distributions. If any of your phishing features are categorical (protocol type, TLD category, etc.), KS-test wouldn't be the appropriate drift test for those columns — you'd want **Chi-Square test** or **Population Stability Index (PSI)** instead. This is a legitimate limitation worth naming proactively if your feature set includes categorical columns.

Q10. "What happens when drift is detected — does the pipeline stop, alert, or auto-retrain?" A. Be honest based on your actual `data_validation.py` behavior. Most likely: it's flagged and stored/logged, but there's no automatic retraining trigger. If asked "what would you add for production," a good answer: an alerting hook or a scheduled retraining trigger conditioned on the drift flag.

Q11. "What other checks does Data Validation perform besides drift?" A. Schema validation — checks the number of columns matches an expected schema defined in `schema.yaml`, and confirms expected numerical columns are present, before data is allowed to proceed to transformation.

4. DATA TRANSFORMATION QUESTIONS

Q12. "Why KNN Imputer instead of mean/median imputation?" A. Mean/median imputation ignores relationships between features — it plugs in one global value regardless of context. KNN Imputer estimates a missing value based on the values of the most similar rows across other features, generally preserving feature relationships better, especially when missingness isn't purely random.

► Will dig in:

- "Computational cost trade-off?" → KNN imputation is more expensive than mean/median since it computes distances across rows; this matters more as dataset size grows.
- "How did you choose k?" → Check your actual `params`/constants value before the interview; if untuned, say you'd tune it via cross-validation.
- "Why not IterativeImputer/MICE?" → Reasonable alternative; you chose KNN for simplicity given a moderate feature-dimensionality dataset, though iterative imputation could model feature relationships even better at higher compute cost.

Q13. "Why remap the label from -1 to 0?" A. This dataset encodes labels as **-1 (phishing)** and **1 (legitimate)**, which isn't the conventional binary format (0/1) most sklearn classifiers and metrics expect. Remapping -1 → 0 ensures downstream metrics (F1, precision, recall) and model outputs are interpreted correctly as standard binary classification.

► **Will dig in:** "Which class is 1 and which is 0 after remapping, and does that matter for precision/recall interpretation?" — Know clearly which class you treat as "positive" (this changes what precision/recall actually mean — e.g., if phishing = 1 = positive class, then recall reflects "% of actual phishing sites correctly caught," which is the metric that matters most for a security use case).

Q14. "What does `np.c_` do when recombining your transformed arrays?" A. It concatenates arrays column-wise — combining the transformed feature columns with the target column back into a single array for storage, so the saved `.npy` file contains both X and y together in one structure.

5. MODEL-BY-MODEL DEEP DIVE — Why & Where Each Is Used

Your pipeline benchmarks 5 classifiers. Here's the "why this model, where it's typically used" explanation for each — a very likely direct interview ask.

Random Forest — ★ Your Winning Model (F1: 0.9711)

- **What it is:** An ensemble of many decision trees, each trained on a random subset of data/features (bagging), with final prediction by majority vote.
- **Why used here:** Handles nonlinear feature interactions well, robust to outliers and irrelevant features, requires little preprocessing, and generally performs strongly on structured/tabular data like URL/network features without heavy tuning.
- **Where it's typically used:** Structured/tabular classification problems broadly — fraud detection, credit scoring, medical diagnosis from tabular features — anywhere you have a mix of feature types and want strong out-of-the-box performance with resistance to overfitting (compared to a single decision tree).
- **Why it likely won here:** Averaging many trees reduces the variance/overfitting risk that a single Decision Tree has, while still capturing the same nonlinear feature splits — a natural fit for tabular security-feature data.

Decision Tree — Runner-up (F1: 0.9649)

- **What it is:** A single tree of if/else splits on feature values, chosen to maximize information gain (or minimize Gini impurity) at each split.
- **Why used here:** Simple, interpretable baseline — you can literally trace the decision path for any prediction, which has real value in security contexts where explainability matters (e.g., "why was this URL flagged").
- **Where it's typically used:** Anywhere interpretability is prioritized alongside decent accuracy, or as a building block for ensemble methods (Random Forest, Gradient Boosting are literally built from many decision trees).
- **Why it's close behind Random Forest:** A single tree is more prone to overfitting the training data's specific patterns, hence a marginally lower generalization score than the ensembled version.

Gradient Boosting (F1: 0.9635)

- **What it is:** Ensemble method that builds trees sequentially, where each new tree corrects the errors (residuals) of the previous ones, combined additively.
- **Why used here:** Often achieves very strong accuracy on structured data by focusing learning capacity on the hardest-to-classify examples; a natural model to benchmark against Random Forest for tabular classification.
- **Where it's typically used:** Kaggle-style tabular competitions, click-through-rate prediction, risk scoring — anywhere marginal accuracy gains matter and some added training time/complexity is acceptable.
- **Trade-off worth naming:** More prone to overfitting than Random Forest if not carefully tuned (learning rate, number of estimators), and slower to train since trees are built sequentially rather than in parallel.

Logistic Regression (F1: 0.9352 — untuned, empty param grid)

- **What it is:** A linear model that estimates the probability of the positive class via a sigmoid function over a weighted sum of input features.
- **Why included here:** Serves as your **baseline** — if a simple linear model gets you most of the way, it questions whether the added complexity of tree ensembles is worth it; if it underperforms significantly, that's evidence the true relationship in the data is nonlinear.
- **Where it's typically used:** Any problem where interpretability of feature weights matters (e.g., "which factors drive this prediction and by how much"), and as a fast, cheap baseline before reaching for heavier models.
-  **Be ready for:** "Why is its param grid empty (`{}`) in your code?" — Honest answer: *"I didn't tune it — it ran with sklearn's default parameters, purely as a simple baseline comparison point against the more complex models."*

AdaBoost (F1: 0.9320 — your weakest performer)

- **What it is:** Boosting method that trains a sequence of weak learners (typically shallow decision trees/"stumps"), where each subsequent learner focuses more weight on previously misclassified examples.
- **Why included here:** Another boosting-family benchmark alongside Gradient Boosting, to compare boosting approaches against bagging (Random Forest).
- **Where it's typically used:** Historically popular for binary classification with relatively clean data; less commonly the top choice today compared to Gradient Boosting/XGBoost variants, but still a solid, fast-training benchmark.
- **Why it likely came in last here:** AdaBoost is more sensitive to noisy data and outliers than Random Forest, since misclassified points (which could include noise) get progressively upweighted — if your phishing dataset has any label noise or ambiguous edge cases, this would disproportionately hurt AdaBoost.

► **Cross-model question to expect:** "If you had to pick just 2 of these 5 to keep, which would you drop and why?" — Good answer: keep Random Forest (best performer) and Logistic Regression (fast, interpretable baseline for sanity-checking); the remaining three are largely redundant with Random Forest's ensemble-tree approach without adding enough marginal value to justify maintaining all of them in production.

6. THE $R^2 \rightarrow F1$ BUG — FULL STORY, RESOLVED

This is now one of the strongest, most concrete parts of your interview story — a real bug you found, understood, fixed, and validated. Have this narrative ready verbatim.

Q15. "Walk me through how you selected the best model — what metric did you use?" A. *"Initially, my `evaluate_models()` utility function was scoring candidate models using `r2_score` — which I discovered, on reviewing my own code, is a **regression metric**, not appropriate for a binary classification task. It still produced a number for each model (0.88, 0.84, 0.82, 0.68, 0.67 across the 5 models) since R^2 is mathematically computable on any numeric input, but that number didn't actually represent meaningful classification performance. I traced this to a leftover import — `from sklearn.metrics import r2_score` — likely copy-pasted from a regression-project template, since the file also had this same stray import at the top I never used elsewhere. I replaced it with `f1_score`, matching the same metric I was already using elsewhere in the pipeline (`get_classification_score()`), reran the comparison, and got much higher, internally consistent scores: Random Forest 0.9711, Decision Tree 0.9649, Gradient Boosting 0.9635, Logistic Regression 0.9352, AdaBoost 0.9320 — which now line up closely with the F1 scores I was already seeing in my MLflow-tracked train/test runs (0.97-0.99 range) for the winning model."*

Q16. "Did fixing the metric change which model won?" A. *"No — Random Forest was the best performer both before and after the fix, and the relative ranking of all 5 models stayed the same. What changed was that the scores became actually meaningful and trustworthy, rather than just happening to preserve the same order by coincidence. That's an important distinction: the bug didn't invalidate my final model choice, but it did mean I had no reliable way to explain or defend those numbers until I fixed it."*

Q17. "Why is F1 a better choice than accuracy or R^2 here specifically?" A. R^2 doesn't apply to classification at all — it measures variance explained in continuous outputs, so using it on 0/1 class predictions produces numbers that don't mean anything in classification terms. Accuracy would be a valid classification metric, but can be misleading on imbalanced datasets (a model predicting the majority class every time can still score high accuracy). F1 — the harmonic mean of precision and recall — is more robust in the presence of imbalance and better reflects a security use case where both catching phishing sites (recall) and not over-flagging legitimate ones (precision) matter.

Q18. "How did you verify your fix was actually correct, not just 'different'?" A. I cross-checked the new comparison scores against F1 scores I already had from a separate, independently-computed source in the pipeline — the MLflow-tracked train/test metrics

for the winning model (computed via `get_classification_score()`, a different function entirely). Before the fix, my comparison scores (R²-based) didn't align with those independently tracked F1 numbers at all. After the fix, they landed in the same range, which gave me confidence the fix was both correct and consistent across the codebase, not just superficially "higher."

Q19. "Is there anything else in your codebase that might have the same kind of copy-paste leftover bug?" A. Good self-critical answer: *"That's exactly the kind of thing I'd want to audit more broadly — I found this one because I was specifically reviewing the model comparison logic, but a full pass checking every import and metric usage against its actual purpose would be a reasonable next step before calling this production-ready."*

7. MLFLOW / DAGSHUB EXPERIMENT TRACKING QUESTIONS

Q20. "Why did your Dagshub Experiments tab originally show only 2 runs, and why does it show 7 now?" A. Originally, `track_mlflow()` was called exactly twice per training run — once logging the winning model's metrics on the training set, once on the test set. That's the same model evaluated twice, not two different models. I extended this by adding a loop that logs **all 5 candidate models'** comparison scores as separate MLflow runs (named `{model_name}_comparison`), so now the Experiments tab shows those 5 additional entries alongside the original 2 train/test runs for the winner — 7 total.

Q21. "Why does this matter — what's the actual benefit of logging all 5, not just the winner?" A. Auditability and reproducibility — without it, there's no historical record of how close the runner-up models were, or whether a different model might have won on a different data split/run. Logging every candidate gives visibility into the full comparison, not just the final decision, which matters if the "winner" needs to be revisited later (e.g., after a retraining cycle where a different model might win).

Q22. "What's Dagshub adding on top of plain local MLflow?" A. Dagshub hosts the MLflow tracking server remotely, so you don't need to self-host/manage one, and it ties experiment tracking directly to your Git repository and commits, giving a shared, persistent dashboard others (or future-you) can access without needing local access to your machine.

8. FASTAPI / DEPLOYMENT / CI-CD QUESTIONS

Q23. "Walk me through your CI/CD pipeline end to end." A. GitHub Actions triggers on push → builds a Docker image from the Dockerfile → pushes that image to AWS ECR → a self-hosted runner installed on my EC2 instance pulls the new image and redeploys the container. That maps to Continuous Integration (build), Continuous Delivery (push to registry), and Continuous Deployment (redploy on the live server).

Q24. "Why a self-hosted runner instead of GitHub's default hosted runners?" A. Deployment needs to happen *inside* my EC2 instance/network — a GitHub-hosted runner

can't directly push files or restart containers on a private EC2 box without significant extra networking/access setup. Installing the runner directly on EC2 lets the deployment step execute locally on the target machine itself.

Q25. "What does `/predict` actually do end to end?" A. Accepts an uploaded CSV via FastAPI's `UploadFile`, loads it into a Pandas DataFrame, loads the saved preprocessor (`preprocessor.pkl`) and trained model (`model.pkl`) via a `NetworkModel` wrapper, runs predictions, appends a `predicted_column`, saves the result as `prediction_output/output.csv`, and also renders the same data as an HTML table via Jinja2 templates for immediate visual feedback in the browser.

Q26. "How would you scale `/predict` for high concurrent load?" A. Move heavy inference off the synchronous request path into a background task or queue (Celery, or a managed queue like SQS), scale horizontally behind a load balancer, and avoid blocking the request-response cycle on model inference for large uploaded files.

Q27 (found earlier, still worth having ready): "I noticed `save_object(..., obj=NetworkModel)` — is that saving the class or the instance?" A. *"Good catch — that's a bug I found reviewing my own code. It's saving the class reference (`NetworkModel`) rather than the instantiated object (`Network_Model`, lowercase) that actually wraps the trained model and preprocessor together. The correct line should reference the lowercase instance. I'd fix that before considering this production-ready — as written, loading that pickle back wouldn't give you a usable, fitted model."*

9. ADVERSARIAL / STRESS-TEST QUESTIONS

Q28. "Convince me your 97% F1 score isn't just an artifact of an easy/synthetic dataset." A. Honest answer: *"That's a fair challenge — many public phishing datasets are somewhat clean/curated compared to live traffic, so a high F1 here reflects strong performance on this benchmark, not a guarantee of real-world performance on messier, adversarial, real-time traffic. I'd want to validate against a more recent, real-world sample before trusting this number in production, and I'd expect some degradation once faced with newer phishing techniques not represented in this training data — which ties back to why the Data Validation drift-detection stage matters for catching that kind of degradation over time."*

Q29. "What's this model's Achilles' heel — when would it likely fail?" A. Novel phishing techniques not represented in the training distribution (e.g., a brand-new URL obfuscation trick), and adversarial URLs specifically crafted to evade known feature patterns — no amount of tuning beats a lack of representative training signal for genuinely new attack patterns. This is also exactly why drift detection and periodic retraining matter operationally, not just at initial training time.

Q30. "Is your dataset balanced or imbalanced between phishing and legitimate classes? Why does that matter for your metric choice?" A. Check your actual class distribution before the interview (`df[target_column].value_counts()`). If it's imbalanced (common in

phishing datasets), that directly justifies why F1 (or even better, F1 with attention to recall specifically) was the right call over raw accuracy — a model could get high accuracy by mostly predicting the majority class, which F1 penalizes more honestly than accuracy would.

Q31. "If I gave you a live, real-time stream of URLs instead of a static CSV, what would change?" A. You'd need real-time feature extraction (computing URL/network features on the fly, not reading them pre-computed from a dataset), lower-latency inference (the current batch `/predict` design assumes file uploads, not per-request real-time scoring), and much tighter monitoring for drift, since live traffic patterns shift faster than a static benchmark dataset would suggest.

10. NUMBERS YOU MUST HAVE COLD

Fact	Value
Data source	MongoDB Atlas (Cluster0)
Models compared	Random Forest, Decision Tree, Gradient Boosting, Logistic Regression, AdaBoost (5)
Drift detection method	Kolmogorov-Smirnov (KS) test
Missing value strategy	KNN Imputer
Label remap	-1 (phishing) → 0
Bug found	<code>r2_score</code> (regression metric) used to score classification models
Fix applied	Replaced with <code>f1_score</code>
Random Forest F1 (winner)	0.9711
Decision Tree F1	0.9649
Gradient Boosting F1	0.9635
Logistic Regression F1	0.9352
AdaBoost F1	0.9320
Experiment tracking	MLflow + Dagshub — 7 total runs (5 comparison + 2 train/test for winner)
Deployment	Docker → AWS ECR → EC2 via self-hosted GitHub Actions runner

Fact	Value
API framework	FastAPI (<code>/train</code>), (<code>/predict</code>)

FINAL NOTE

The single strongest asset you now have on this project is **the $R^2 \rightarrow F1$ story** — it's a real bug, you understood *why* it was wrong (not just that it "seemed off"), you fixed it, and you cross-validated the fix against an independent source (your MLflow-tracked scores) rather than just accepting the new numbers on faith. That arc — **found it $\rightarrow$ understood it $\rightarrow$ fixed it $\rightarrow$ verified it** — is exactly what separates "I followed a tutorial" from "I understand what I built." Lead with this story whenever the conversation opens up room for it, even if not asked directly.